CLEAR-MoE: Shared-Basis Expert Extraction from Frozen Vision Transformers via Calibration-Driven Layer Selection

***Abstract*—We present CLEAR-MoE, a four-phase post-training pipeline that converts a frozen pretrained vision transformer (ViT) into a sparse mixture-of-experts (MoE) model without updating backbone weights. The pipeline (i) scores FFN layers by sparsity, clusterability, and output sensitivity; (ii) decomposes selected layers into a shared low-rank SVD basis plus per-cluster residual experts (k-means); (iii) fits lightweight routers supervised by cluster labels; and (iv) dispatches tokens via pluggable CUDA backends. On Imagenette with DeiT-Small, CLEAR-MoE retains 99.9% of dense accuracy (86.70 ± 0.02% vs. 86.73%). Our ablations isolate a consistent empirical finding: the shared SVD basis is the dominant accuracy-preserving component. Random routing, learned routing, and three router architectures yield numerically similar results spanning at most 0.06 pp (86.62–86.68%), and accuracy remains stable across SVD rank, expert count $E \in \{2, \ldots, 8\}$, calibration size $N \in \{50, \ldots, 500\}$, and random seed. This finding generalizes across five ViT backbones (DeiT-T/S/B, ViT-S/B, 5.7M–86.6M parameters), with $|\Delta| \leq 0.10$ pp across all configurations. On a GTX 960, routing and scatter-gather overhead make CLEAR-MoE FFN 1.3–1.7× slower than dense. A dispatch microbenchmark indicates that routing (AI = 1.9 FLOPs/B) is an order of magnitude more memory-bound than expert GEMMs (AI = 22.7), identifying fused dispatch kernels as a plausible optimization target.**

***Index Terms*—mixture of experts, vision transformer, post-training extraction, shared basis decomposition, calibration-driven scoring, dispatch strategies, roofline analysis**

I. Introduction

Vision transformers (ViTs) [1] devote roughly two-thirds of inference FLOPs to feed-forward network (FFN) sub-layers that execute the same MLP computation for every token, regardless of semantic content [3]. Sparse mixture-of-experts (MoE) architectures [2] address this by routing each token to a subset of specialized sub-networks, reducing active compute proportional to k/E. However, existing vision MoE models [3], [6] require training from scratch, which is a resource barrier for practitioners who already hold a pretrained checkpoint.

Post-training expert extraction converts a dense FFN into experts without retraining [8]–[10]. Despite recent progress, three gaps remain: (1) most methods convert all FFN layers, ignoring sensitivity variation across depth; (2) disjoint expert assignment discards shared low-level structure, destabilizing downstream transfer; (3) speedup claims are reported in MACs rather than wall-clock latency on real hardware.

CLEAR-MoE addresses all three with the following contributions:

- **Calibration-driven layer scoring**: composite score $\mathcal{S}(l) = 0.4\,\text{sparsity} + 0.4\,\text{clusterability} - 0.2\,\text{sensitivity}$ selects FFN layers that cluster well and tolerate perturbation, automatically excluding high-sensitivity layers (e.g., block 0, sensitivity = 0.946).
- **Shared-basis decomposition**: fc2 is decomposed into a shared truncated-SVD basis plus per-cluster residuals; fc1 is shared identically across all experts. This preserves common visual structure while allowing per-cluster specialization.
- **Hardware-transparent latency study**: all timing on a single GTX 960 (4 GB, 112 GB/s) with `cuda.synchronize()`-fenced p50 reporting (100 forward passes), indicating that routing overhead (not expert arithmetic) is a likely bottleneck.
- **Comprehensive ablation**: decomposition (D0–D8), layer selection (L0–L10), dispatch backends, expert count, SVD rank, router architecture, calibration size, and random seed, establishing which design choices actually matter.

Scope. All measured results use a single GTX 960. No claims are made about data-center GPUs, distributed setups, or larger datasets. Multi-device throughput projections are modelled analytically using PCIe Gen3×16 bandwidth; no NCCL runs were performed.

II. Related Work

Post-training expert extraction. MoEfication [10] first showed that clustering neuron activation patterns partitions a pretrained FFN into experts without retraining. D2DMoE [8] extended this to dynamic-k routing, achieving 30% latency reduction on an A100 for ViT-B/ImageNet-1K. Berisha et al. [9] used variance-based neuron grouping to recover 98% of dense performance with 36.3% MAC reduction for DeiT-B. CLEAR-MoE differs by (a) selecting layers via a composite sensitivity-aware score, (b) preserving shared structure via SVD decomposition, and (c) providing a full wall-clock dispatch study on consumer hardware. Table I positions these methods side-by-side. Sparse Upcycling [14] converts dense language-model checkpoints to MoE by cloning FFN weights and training a load-balancing router via continued pre-training; unlike CLEAR-MoE, backbone weights are updated, requiring gradient access and sufficient unlabelled data.

Vision MoE training. V-MoE [3] demonstrated that sparse patch routing matches dense quality at ∼50% active compute for 15B-parameter ViTs. AdaMV-MoE [6] showed late transformer layers benefit most from expertization, a heuristic

TABLE I

POST-TRAINING MOE EXTRACTION: METHOD COMPARISON. † = MAC REDUCTION (NOT WALL-CLOCK). ‡ = MINIMAL FINE-TUNING REPORTED BY ORIGINAL PAPER. CLEAR-MOE: ZERO BACKBONE WEIGHT UPDATES.

Method	Backbone	HW	Acc. Ret.	Latency	Expert Struct.
MoEfication [10]	ViT	N/A	∼99%	N/A	Disjoint k-means
D2DMoE [8]‡	ViT-B/16	A100	∼99%	−30%	Dynamic-k disjoint
Berisha [9]‡	DeiT-B	N/A	98%	−36.3%†	Variance-based disjoint
CLEAR-MoE (ours)	**DeiT-T/S/B, ViT-S/B**	**GTX 960**	**99.9%**	+1.3–1.7×	**Shared SVD + residuals**

D2DMoE latency on A100 (2 TB/s); CLEAR-MoE on GTX 960 (112 GB/s). Bandwidth 18× lower.

our composite score can override when sensitivity signals contradict it. DynamicViT [4] and A-ViT [5] achieve token-level sparsification via halting gates, complementary to our FFN-level approach.

MoE runtime systems. TUTEL [11] achieves 3.11× speedup via 2D all-to-all on 128 GPUs. Brainstorm [12] shows profile-guided dispatch yields 5× speedup over DeepSpeed for SwinV2-MoE. Lancet [13] reduces non-overlapping communication by 77% via whole-graph overlap. These systems target high-bandwidth multi-GPU clusters; CLEAR-MoE studies the single-consumer-GPU regime where bandwidth is 18× lower.

III. METHOD

CLEAR-MoE converts a frozen pretrained ViT into a selectively expertised model through four sequential phases (Fig. 1). No backbone weights are updated.

A. Phase 1: Calibration Pass

Forward N_{cal} representative images through the frozen model, recording pre-FFN activation tensors at each of the L selected layers via PyTorch hooks. For DeiT-Small (d=384, N_{cal}=200, 197 tokens/image), this produces a [39,400, 384] activation matrix per layer.

B. Phase 2: Layer Scoring and Selection

Each FFN layer l receives a composite score:

$$\mathcal{S}(l) = 0.4\,\text{sparsity}(l) + 0.4\,\text{clusterability}(l) - 0.2\,\text{sensitivity}(l) \quad (1)$$

where *sparsity* is the fraction of activation magnitudes below 0.01; *clusterability* is the Silhouette score of k-means with k=E on the calibration activations; and *sensitivity* is the normalized logit change when the layer's FFN output is zeroed. Sensitivity is *subtracted* because high sensitivity implies high degradation risk if expertized. The top-k=$L/2$ layers by $\mathcal{S}$ are selected; the remaining layers retain their original dense FFN. Table II reports per-layer scores for DeiT-Small (200-image calibration); block 0 ranks last due to sensitivity = 0.946.

C. Phase 3: Expert Extraction

For each selected layer, fc2 ($W_2 \in \mathbb{R}^{d\times d_{\text{ffn}}}$) is decomposed via truncated SVD; fc1 is shared identically across all experts:

$$y = \underbrace{W_{\text{shared}}}_{\text{SVD-}r\text{ approx.}} \cdot z + \underbrace{W^{\text{res}}_{e^*}}_{\text{cluster residual}} \cdot z, \quad z = \text{GELU}(\text{fc1}(x)) \quad (2)$$

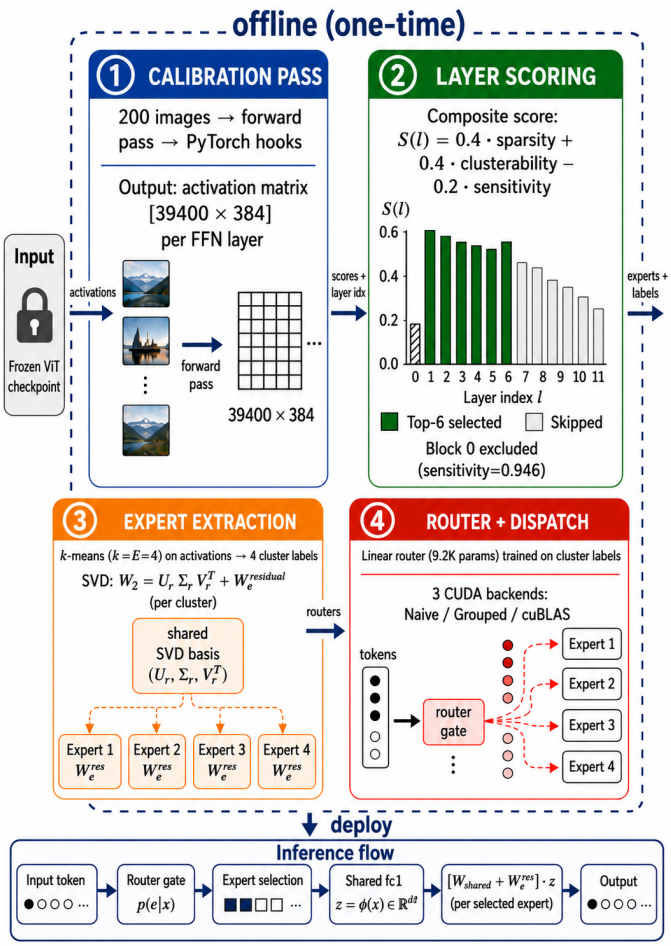

Fig. 1. CLEAR-MoE pipeline. A frozen pretrained ViT yields calibration activations; scored FFN layers are decomposed into a shared SVD basis plus per-cluster residual experts; lightweight routers are fitted on k-means labels; inference uses pluggable CUDA dispatch backends.

TABLE II

PER-LAYER COMPOSITE SCORES, DEIT-SMALL (200-IMAGE CALIBRATION). COMPOSITE = 0.4·SP + 0.4·CL − 0.2·SE. BOLD = SELECTED (TOP-k=6).

Block	Sparsity	Clusterab.	Sensitivity	Composite
blocks.1	0.180	0.526	0.171	**0.248**
blocks.4	0.152	0.518	0.109	**0.246**
blocks.5	0.152	0.520	0.113	**0.246**
blocks.3	0.149	0.519	0.113	**0.245**
blocks.6	0.147	0.522	0.123	**0.243**
blocks.2	0.141	0.518	0.147	**0.234**
blocks.7	0.147	0.519	0.162	0.234
blocks.11	0.134	0.516	0.149	0.230
blocks.9	0.135	0.518	0.167	0.228
blocks.8	0.142	0.512	0.175	0.227
blocks.10	0.171	0.514	0.252	0.224
blocks.0	0.210	0.516	0.946	0.101

$W_{\text{shared}} = U_r \Sigma_r V_r^\top$ retains the top-r singular values (default r = 50% of matrix rank, r=$d/2$=192 for DeiT-Small where $W_2 \in \mathbb{R}^{384\times 1536}$ has maximum rank 384; materialised as a

dense matrix before inference, with no factorised compute reduction). k-means with $k{=}E$ partitions calibration tokens into E clusters; the per-cluster residual is $W_e^{\text{res}} = (W_2 - W_{\text{shared}}) \cdot s_e$, where s_e scales by the ratio of cluster-mean to global-mean activation norm. Because s_e is a scalar, all residual experts share the same directional component $(W_2 - W_{\text{shared}})$, differing only in magnitude; a misrouted token therefore receives a different scaling but the same directional correction. During inference, z is computed once and reused for both the shared and residual paths.

D. Phase 4: Router Fitting

A router g_θ predicts the cluster assignment of each token. The k-means labels from Phase 3 provide supervised targets: router training minimizes cross-entropy between $g_\theta(x)$ and cluster label c over the calibration tokens, using AdamW (lr$=10^{-3}$, 5 epochs, cosine decay). Three router architectures are evaluated: **Linear** ($d \times E$ parameters, 9.2K total), **MLP** (hidden layer $d/6$, 149K), and **Adaptive** (confidence-threshold top-1/top-2 escalation, same parameter count as Linear).

E. Dispatch Backends

Three CUDA backends are implemented: (1) **Naive**: boolean-mask loop over experts (reference). (2) **Grouped**: tokens are sorted by expert index; one GEMM per contiguous sub-batch. No padding; throughput stable under imbalance. (3) **cuBLAS**: expert sub-batches are padded to the largest and stacked into a 3D tensor dispatched to a batched GEMM call. Fastest at balanced load; degrades at high imbalance from padding waste.

IV. Experiments

Setup. DeiT-Small (22M parameters, $d{=}384$, $d_{\text{ffn}}{=}1536$, 12 blocks) evaluated on Imagenette (3,925 validation images, 10-class ImageNet subset) with 200-image calibration set, $E{=}4$ experts, AdamW router training (5 epochs, lr 10^{-3}, cosine decay). All timing: p50 of 100 forward passes, batch size 8, GTX 960 (4 GB, 112 GB/s), `cuda.synchronize()`-fenced.

A. Decomposition Ablation (D0–D8)

Table III ablates what each architectural choice contributes to accuracy and latency. Configurations D0–D8 span the design space from dense baseline to shared-basis CLEAR-MoE FFN, isolating the effect of the shared basis, the routing mechanism, and the expert assignment strategy.

The shared SVD basis is the dominant accuracy-preserving component. D3 (shared basis + full global residual, no routing) achieves exact reconstruction: algebraically, $W_{\text{shared}} \cdot z + (W_2 - W_{\text{shared}}) \cdot z = W_2 \cdot z$, matching D0 at 86.73%. D2 and D7 (disjoint experts, no shared fc1) collapse to 65–66% regardless of routing strategy: disjoint expert weights estimated from 200 calibration images fail to generalize. In contrast, D4/D5 (shared basis, *random* routing) retain 86.62–65%, only 0.08–0.11 pp below D0; D6/D8 (shared basis, learned routing at 95% accuracy) achieve an identical

TABLE III

Decomposition ablation (DeiT-Small, Imagenette, $E{=}4$, last-k layers, batch=8, GTX 960). ΔTop-1 relative to D0. D4/D5 differ only in random seed. D6 and D8 are the same architecture: D6 is the cold-kernel first dispatch; D8 is measured after 3 warm-up passes (steady-state). The 21 ms gap reflects CUDA JIT overhead on first run.

ID	Shared fc1	Shared fc2	Router	Top-1	Δpp	p50 ms
D0	–	–	None (dense)	86.73%	+0.00	59.6
D1	✓	SVD rank-r	None	86.55%	−0.18	59.7
D2	✗	Disjoint	Random	66.42%	−20.31	49.4
D3	✓	Full residual	None	86.73%	+0.00	59.9
D4/D5	✓	k-means res.	Random	86.62–65%	−0.08–11	92.2
D6/D8	✓	k-means res.	Linear (learned)	86.65%	−0.08	78.8–100.3
D7	✗	Disjoint	k-means	65.22%	−21.51	48.0

✗ = disjoint (no shared fc1); ✓ = shared across all tokens.

-0.08 pp. Routing quality has limited measured impact once the shared basis is present.

No latency reduction on GTX 960. CLEAR-MoE FFN (D4–D8) executes shared fc1+fc2 on *all* tokens plus residual paths that collectively process all T tokens (T/E per expert $\times$ E experts), adding approximately one full fc2-equivalent computation; total arithmetic is $\approx 1.5\times$ dense FFN. On bandwidth-limited memory (112 GB/s), this incurs 1.3–1.7$\times$ latency overhead (78–100 ms vs. 59.6 ms). Only disjoint experts (D2/D7) are faster (48–49 ms) by eliminating the shared path entirely, at the cost of 21 pp accuracy.

B. Layer-Selection Ablation (L0–L10)

Table IV compares all 11 layer-selection strategies on the same MoE configuration ($E{=}4$, $k{=}6$ of 12 FFN layers, composite scoring).

All 11 policies span only 0.21 pp (86.62–86.83%), confirming that *accuracy is policy-insensitive*. The shared basis preserves quality regardless of which 6 blocks are expertized. The composite score's practical value is principled block 0 exclusion (sensitivity = 0.946): L2 (first-k) includes block 0 and incurs the worst -0.13 pp drop. L10 avoids it, achieving 82.4 ms, comparable to L3 (last-k, 80.3 ms) while additionally excluding high-sensitivity blocks. L7 (high-sensitivity, deliberately selecting the most sensitive blocks as a stress-test baseline) achieves only -0.08 pp accuracy loss but is the slowest policy at 102.5 ms, demonstrating that layer choice affects latency more than accuracy.

C. Dispatch Strategy Benchmark

We benchmark three CUDA backends ($B{=}8$, $T{=}1568$, $E{=}4$, GTX 960) under four imbalance levels (Table V). At balanced load, cuBLAS peaks at 728 K tok/s (2.79$\times$ CPU serial). At 80% imbalance, padding waste collapses cuBLAS to 558 K tok/s (-23%). Grouped dispatch remains stable at 665–747 K tok/s across all imbalance levels. **Recommendation**: cuBLAS for predictably balanced load; Grouped otherwise.

D. Roofline Analysis

Fig. 2 places each CLEAR-MoE operation on the GTX 960 roofline (peak FP32: 2.4 TFLOP/s, BW: 112 GB/s, ridge: 21.4 FLOPs/B). Dense FFN (AI = 74.3) and expert GEMMs

TABLE IV
COMPLETE LAYER-SELECTION ABLATION (L0–L10). ALL: DEIT-SMALL, E=4, k=6 LAYERS, LINEAR ROUTER, 200-IMAGE CALIBRATION. L1 AVG OF 3 SEEDS. Δ RELATIVE TO L0.

ID	Strategy	Layers	Top-1	Δpp	p50 ms
L0	Dense (no MoE)	–	86.73%	+0.00	57.0
L1	Random (n=3)	varies	86.72±0.01%	−0.01	78.5±0.8
L2	First k	0–5	86.60%	−0.13	79.3
L3	Last k	6–11	86.68%	−0.05	80.3
L4	Alternating (odd)	1,3,5,7,9,11	86.83%	+0.10	87.2
L5	Sparsity only	0,1,3,4,5,10	86.73%	+0.00	99.1
L6	Clusterability	1,2,3,5,6,7	86.73%	+0.00	88.8
L7	High-sensitivity	0,1,7,8,9,10	86.65%	−0.08	102.5
L8	Sp.+Cl.	0,1,4,5,6,10	86.62%	−0.10	88.8
L9	Cl.−Se.	2,3,4,5,6,11	86.68%	−0.05	94.0
L10	**Composite (ours)**	**1–6**	**86.68%**	**−0.05**	**82.4**

TABLE V
DISPATCH MICRO-BENCHMARK: 0% VS. 80% IMBALANCE (B=8, T=1568, E=4, GTX 960).

	0% imb.		80% imb.	
Backend	ms	K tok/s	ms	K tok/s
CPU Serial	6.01	261	5.96	263
Naive	3.67	428	3.59	437
Grouped	2.36	**665**	2.14	**733**
cuBLAS	**2.15**	**728**	2.81	558

(AI = 22.7) are compute-bound. The router gate (AI = 1.9) and token sort (AI = 1.0) are *both* deep in the memory-bound regime: 11× and 21× below the ridge point. This indicates that routing and token sort are strongly memory-bound (11–21× below the ridge point), making fused dispatch kernels a plausible high-leverage optimization. Expert GEMMs are already near the compute ceiling and offer diminishing returns.

E. Hyperparameter Sensitivity

We ablate SVD rank, expert count, router architecture, calibration size, and random seed (Table VI); findings are:

SVD rank ($r \in \{16, 32, 64, 96, 128, 192, 256\}$): accuracy range is 86.62–86.75% (0.13 pp). Reconstruction error decreases monotonically from 0.91 to 0.29, but this improvement does not translate to accuracy. Even r=16 captures the shared basis sufficiently.

Expert count ($E \in \{2, 4, 8, 16\}$): accuracy is 86.65–86.80% for $E \in \{2, 4, 8\}$; E=16 drops 0.26 pp as 200 calibration images provide insufficient statistics to estimate 16 fine-grained clusters. Latency grows ∼9 ms per doubling of E (73 ms at E=2 to 98 ms at E=16). No empty experts appear at any count.

Router architecture: all three routers (Linear, MLP, Adaptive) achieve identical 86.68% Top-1. MLP routing accuracy reaches 0.980 vs. 0.925 for Linear, a 5.5 pp improvement in routing precision with zero downstream benefit. The gap between random routing (D4/D5, Table III) and all learned routers is at most 0.06 pp (roughly two changed predictions out of 3,925). While this numerically exceeds the 3-seed

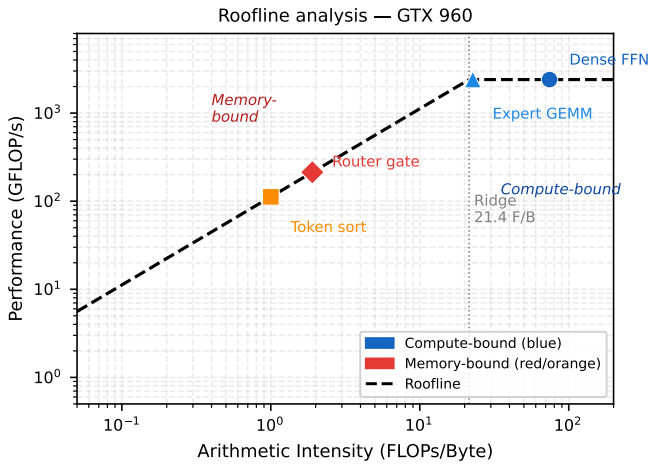

Fig. 2. Roofline for GTX 960. Blue = compute-bound; orange/red = memory-bound. Router gate (AI = 1.9) is 11× below the ridge point (21.4 FLOPs/B); routing, not expert arithmetic, is the primary latency bottleneck.

TABLE VI
HYPERPARAMETER SENSITIVITY SUMMARY (DEIT-SMALL, IMAGENETTE). ALL RANGES ≤0.41 PP.

Study	Range tested	Top-1 span	Δpp
SVD rank r	16–256	86.62–86.75%	0.13
Expert count E	2–16	86.39–86.80%*	0.41
Router arch.	Lin/MLP/Adaptive	**86.68%** (all)	0.00
Calibration N	50–500	86.55–86.70%	0.15
Random seed	42/123/456	86.70±0.02%	0.05

* $E \in \{2, 4, 8\}$ spans 0.15 pp; E=16 drops 0.26 pp (calibration instability: 200 images insufficient for 16 clusters).

standard deviation of ±0.02 pp (Table VI, row "Random seed"), a paired significance test would be needed to confirm a reliable effect; we treat 0.06 pp as practically negligible. Fig. 3 provides a mechanistic view: despite D6 (learned router) reducing mean per-token routing entropy 3.5× versus D5 (random, $\ln 4 \approx 1.386$ nats), the accuracy impact is negligible.

Fig. 4 plots accuracy gap against router training across 13 configurations (D5 at x=0, no trained router); no strong monotonic relationship is visible, consistent with the hypothesis that shared-basis quality governs accuracy regardless of routing quality.

Calibration size ($N \in \{50, 100, 200, 500\}$, multiple subsets for N<500): accuracy spans 86.55–86.70% across all sizes; N=50 yields 86.65–86.70% (within 0.15 pp of N=500). Router accuracy climbs from 0.83 to 0.95 as N grows, without accuracy benefit.

Reproducibility: 86.70 ± 0.02% Top-1 across seeds 42, 123, 456 (full pipeline, composite scoring, E=4, N=200).

F. Cross-Backbone Generalization

Table VII tests whether the shared-basis finding extends across architectures. We apply the full CLEAR-MoE pipeline to five backbones spanning 5.7–86.6 M parameters and two pretraining lineages (DeiT, ViT); all other settings are identical to the DeiT-S ablation (E=4, N=200, seed 42).

Routing differences are numerically small. Learned routing (D6) provides a marginal numerical advantage over ran-

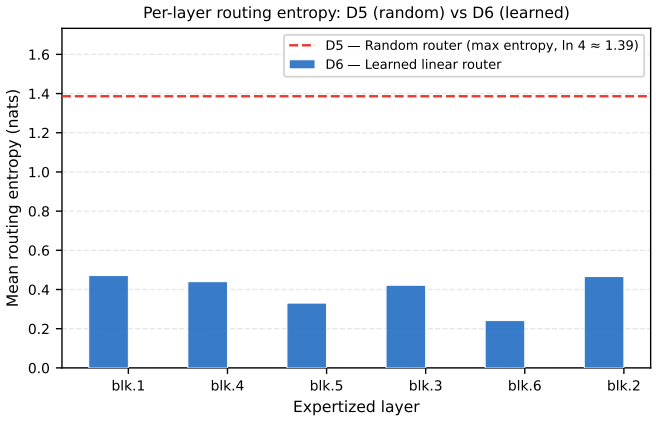

Fig. 3. Mean per-token routing entropy (nats) per expertized layer. Dashed red = D5 (random routing, max entropy $\ln 4 \approx 1.386$). Blue bars = D6 (learned linear router, mean ≈ 0.40 nats). Despite $3.5\times$ entropy reduction, the D5-vs-D6 accuracy gap is only 0.06 pp, confirming that routing confidence does not drive accuracy; the shared SVD basis does.

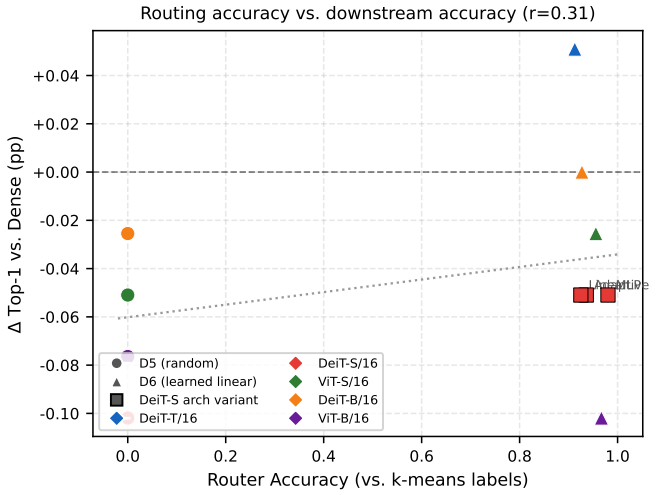

Fig. 4. Accuracy gap (Δpp relative to dense) vs. router training across 13 configurations. D5 (random, no trained router) at $x{=}0$; D6 and cross-backbone routers at measured routing accuracy. No strong monotonic relationship is visible, consistent with the hypothesis that shared-basis quality governs accuracy regardless of routing precision.

TABLE VII
CROSS-BACKBONE GENERALIZATION (IMAGENETTE, $E{=}4$, $N{=}200$, SEED 42). D5 = RANDOM ROUTING; D6 = LEARNED LINEAR ROUTER. $|\Delta| \leq 0.10$ PP ACROSS ALL 10 CONFIGURATIONS.

Backbone	Params	Dense	D5 Δ pp	D6 Δ pp	D6 Rtr. Acc	D6 Skew
DeiT-T/16	5.7 M	75.92%	−0.05	+0.05†	0.913	0.090
DeiT-S/16	22.1 M	86.73%	−0.10	−0.05	0.925	0.099
ViT-S/16	22.1 M	76.23%	−0.05	−0.03	0.956	0.139
DeiT-B/16	86.6 M	91.77%	−0.03	±0.00	0.927	0.094
ViT-B/16	86.6 M	85.38%	−0.08	−0.10‡	0.967	0.150

†Single-seed result for this backbone; per-backbone seed variance not characterised.
‡Highest load skew (0.150); see Discussion.

dom routing (D5) in 4 of 5 backbones; however, the per-backbone improvements range from 0.02 to 0.10 pp, too small to confirm without paired significance testing. The one exception is ViT-B/16, where D5 (−0.08 pp) outperforms D6 (−0.10 pp): The ViT-B result coincides with the highest observed load skew (0.150, vs. 0.094 for DeiT-B), which may contribute to the difference; no ablation was performed to confirm a causal link. Taken together, these results are consistent with the hypothesis that routing quality has limited impact once the shared SVD basis is present; a definitive claim would require paired prediction-level testing across additional seeds. Fig. 5 visualises all 10 configurations.

V. DISCUSSION

Why routing differences are small. All residual experts are scalar-conditioned variants of the same matrix $(W_2 - W_{\text{shared}})$, so mis-routing changes only the scaling factor applied to a token's correction, not its direction. The 0.06 pp gap between random and learned routing is the direct consequence. Cross-backbone results (Table VII) are consistent across 4 of 5 architectures; a linear router (<10K parameters) appears adequate. Expert Choice routing [15], where experts select their tokens rather than the reverse, achieves better load balance in language models; on DeiT-S, however, random and learned routing differ by at most 0.06 pp, and all cross-backbone random-routing losses remain within 0.10 pp of dense, supporting the hypothesis that routing strategy is secondary to decomposition quality.

Why CLEAR-MoE is slower on the GTX 960. CLEAR-MoE FFN computes shared fc1+fc2 on *all* tokens, then adds residual paths that each process T/E tokens but collectively span all T tokens across E experts, adding approximately one full fc2-equivalent computation. Total arithmetic is $\approx 1.5\times$ dense FFN. On a bandwidth-limited GPU (112 GB/s vs. 2 TB/s for an A100), loading shared weights and expert-residual weights across memory dominates. The dispatch micro-benchmark isolates this: routing and token sort (AI = 1.0–1.9) are 11–21$\times$ below the compute ridge. Disjoint experts (D2/D7) are faster (48 ms) by eliminating shared paths, but sacrifice 21 pp accuracy. Analytical modelling suggests $\geq$800 GB/s bandwidth may be needed for latency parity; A100 (2 TB/s) or H100 (3.35 TB/s) class hardware are the plausible candidates.

Practical design guidelines. *When to use CLEAR-MoE.* CLEAR-MoE may be well suited when accuracy preservation is non-negotiable and backbone fine-tuning is infeasible: the evaluated DeiT and ViT backbones can be converted with 200 calibration images and no GPU cluster. The method is not competitive with disjoint-expert approaches if latency reduction is the primary goal on consumer hardware.

Choosing E and r. Expert count $E{=}4$ provides a reasonable empirical trade-off: accuracy remains stable from $E{=}2$ to $E{=}8$, and $E{=}4$ balances router overhead with residual expressiveness. For $E{=}16$, our analytical model suggests $\geq$800 calibration images may be needed for stable cluster estimates; this was not directly validated. SVD rank r can be set as low as 16 without consistent measured accuracy reduction

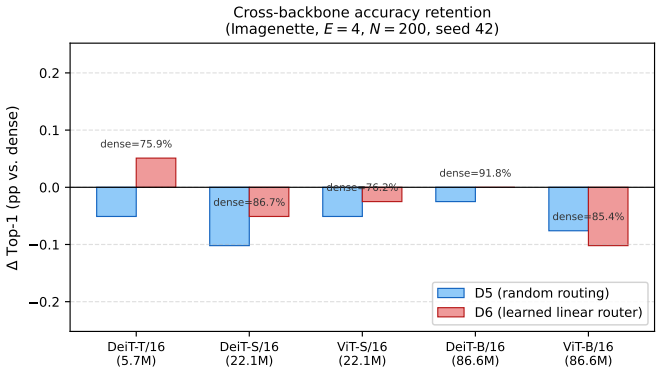

Fig. 5. Accuracy Δ vs. dense for five ViT backbones under D5 (random) and D6 (learned linear) routing (Imagenette, $E{=}4$, $N{=}200$, seed 42). All deltas within $|\Delta|{\leq}0.10$ pp. D6 provides marginal numerical advantage in 4 of 5 backbones; D5 outperforms D6 only on ViT-B (highest load skew, 0.150).

on Imagenette; the default $r{=}d/2{\approx}192$ for DeiT-Small is conservative and may waste compute on deep layers.

Choosing the dispatch backend. cuBLAS is optimal only for perfectly balanced expert load, which is common in classification but rare in detection/segmentation. For workloads with variable token-to-expert ratios, Grouped dispatch is approximately 31% faster than cuBLAS at 80% imbalance and remains comparatively stable across imbalance levels. Naive dispatch is a reference implementation only and is not recommended for latency-sensitive deployment.

Limitations. *Backbone and dataset scope.* DeiT-T/S/B and ViT-S/B generalization is confirmed (Table VII); ViT-L, ImageNet-1K, and hierarchical backbones (Swin, ConvNeXt) are not evaluated. Parameter count, memory footprint, and FLOPs versus a same-hardware baseline are not reported.

Hardware and compute. CLEAR-MoE is 1.3–1.7$\times$ slower than dense on GTX 960 (112 GB/s). Latency parity on high-bandwidth hardware is modelled analytically only; multi-device projections are simulated (PCIe Gen3$\times$16), not measured NCCL runs.

VI. Conclusion

CLEAR-MoE demonstrates that post-training expert extraction preserves $\geq$99.9% of dense ViT accuracy via a shared SVD basis plus per-cluster residuals, requiring only 200 calibration images and a single consumer GPU. This finding is consistent across five ViT backbones spanning 5.7–86.6 M parameters and two pretraining lineages (Table VII), providing preliminary cross-backbone support for the shared-basis hypothesis.

Central finding. The shared SVD basis is the dominant accuracy-preserving component. Routing quality, SVD rank, expert count ($E \in \{2, 4, 8\}$), and calibration size ($N \in \{50, \ldots, 500\}$) are all secondary. Random routing achieves 86.62%, while learned routing at 98% accuracy achieves 86.68%, a 0.06 pp difference whose statistical reliability would require paired testing to confirm. This simplifies the design space: a 10K-parameter linear router appears sufficient when the shared basis is used. Future extraction work should invest in decomposition quality rather than router expressiveness.

Latency finding. On a bandwidth-constrained GTX 960 (112 GB/s), CLEAR-MoE FFN is 1.3–1.7$\times$ slower than dense because the shared basis forces loading shared weights for *all* tokens. Roofline analysis confirms routing and scatter-gather (AI = 1.0–1.9) are 11–21$\times$ below the ridge point while expert GEMMs (AI = 22.7) are near the compute ceiling. Fused dispatch kernels, not expert arithmetic optimizations, are therefore the most plausible high-leverage engineering target.

Future directions. Evaluate CLEAR-MoE on ViT-L and ImageNet-1K to assess scaling beyond the five backbones validated here. Implement Triton-fused router-dispatch kernels to reduce scatter-gather overhead below the memory-bound bottleneck. Extend the composite layer-scoring criterion to hierarchical backbones (Swin, ConvNeXt) where FFN widths are non-uniform across stages. Empirically evaluate CLEAR-MoE on high-bandwidth hardware (A100/H100 class) to determine whether the projected latency parity materialises in practice.

References

[1] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[2] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[3] C. Riquelme et al., "Scaling vision with sparse mixture of experts," in *Proc. NeurIPS*, 2021.

[4] Y. Rao et al., "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. NeurIPS*, 2021.

[5] A. Yin et al., "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. CVPR*, pp. 10809–10818, 2022.

[6] Y. Chen et al., "AdaMV-MoE: Adaptive multi-task vision mixture-of-experts," in *Proc. ICCV*, pp. 17346–17357, 2023.

[7] G. Jain et al., "Mixture of nested experts: Adaptive processing of visual tokens," in *Proc. NeurIPS*, 2024.

[8] F. Szatkowski et al., "Exploiting activation sparsity with dense to dynamic-k mixture-of-experts conversion," in *Proc. NeurIPS*, 2024.

[9] U. Berisha et al., "Efficient data-driven MoE extraction from trained networks," in *Proc. CVPR*, 2025.

[10] Z. Zhang et al., "MoEfication: Transformer feed-forward layers are mixtures of experts," in *Findings of ACL*, pp. 877–890, 2022.

[11] Y. Hwang et al., "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[12] W. Cui et al., "Optimizing dynamic neural networks with Brainstorm," in *Proc. USENIX OSDI*, pp. 759–778, 2023.

[13] C. Jiang et al., "Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping," in *Proc. MLSys*, 2024.

[14] A. Komatsuzaki et al., "Sparse upcycling: Training mixture-of-experts from dense checkpoints," in *Proc. ICLR*, 2023.

[15] Z. Zhou et al., "Mixture-of-experts with expert choice routing," in *Proc. NeurIPS*, 2022.